JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class AbstractSet<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractSet<E>

Type Parameters:

`E - the type of elements maintained by this set`

All Implemented Interfaces:

`Iterable<E>, Collection<E>, Set<E>`

Direct Known Subclasses:

`ConcurrentSkipListSet, CopyOnWriteArraySet, EnumSet, HashSet, TreeSet`

```
public abstract class AbstractSet<E>
extends AbstractCollection<E>
implements Set<E>
```

This class provides a skeletal implementation of the `Set` interface to minimize the effort required to implement this interface.

The process of implementing a set by extending this class is identical to that of implementing a Collection by extending AbstractCollection, except that all of the methods and constructors in subclasses of this class must obey the additional constraints imposed by the `Set` interface (for instance, the add method must not permit addition of multiple instances of an object to a set).

Note that this class does not override any of the implementations from the `AbstractCollection` class. It merely adds implementations for `equals` and `hashCode`.

This class is a member of the Java Collections Framework.

Since:

`1.2`

See Also:

Collection, AbstractCollection, Set

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**AbstractSet**()	Sole constructor.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	**equals**(**Object** o)	Compares the specified object with this set for equality.
int	**hashCode**()	Returns the hash code value for this set.
boolean	**removeAll**(**Collection**<?> c)	Removes from this set all of its elements that are contained in the specified collection (optional operation).

Methods inherited from class java.util.AbstractCollection

add, addAll, clear, contains, containsAll, isEmpty, iterator, remove, retainAll, size, toArray, toArray, toString

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Set

add, addAll, clear, contains, containsAll, isEmpty, iterator, remove, retainAll, size, spliterator, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

AbstractSet

```
protected AbstractSet()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

equals

```
public boolean equals(Object o)
```

Compares the specified object with this set for equality. Returns `true` if the given object is also a set, the two sets have the same size, and every member of the given set is contained in this set. This ensures that the `equals` method works properly across different implementations of the `Set` interface.

This implementation first checks if the specified object is this set; if so it returns `true`. Then, it checks if the specified object is a set whose size is identical to the size of this set; if not, it returns false. If so, it returns `containsAll((Collection) o)`.

Specified by:

`equals` in interface `Collection<E>`

Specified by:

`equals` in interface `Set<E>`

Overrides:

`equals` in class `Object`

Parameters:

`o - object to be compared for equality with this set`

Returns:

`true` if the specified object is equal to this set

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns the hash code value for this set. The hash code of a set is defined to be the sum of the hash codes of the elements in the set, where the hash code of a `null` element is defined to be zero. This ensures that `s1.equals(s2)` implies that `s1.hashCode()==s2.hashCode()` for any two sets `s1` and `s2`, as required by the general contract of `Object.hashCode()`.

This implementation iterates over the set, calling the `hashCode` method on each element in the set, and adding up the results.

Specified by:

`hashCode` in interface `Collection<E>`

Specified by:

`hashCode` in interface `Set<E>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this set

See Also:

`Object.equals(Object)`, `Set.equals(Object)`

removeAll

```
public boolean removeAll(Collection<?> c)
```

Removes from this set all of its elements that are contained in the specified collection (optional operation). If the specified collection is also a set, this operation effectively modifies this set so that its value is the *asymmetric set difference* of the two sets.

This implementation determines which is the smaller of this set and the specified collection, by invoking the `size` method on each. If this set has fewer elements, then the implementation iterates over this set, checking each element returned by the iterator in turn to see if it is contained in the specified collection. If it is so contained, it is removed from this set with the iterator's `remove` method. If the specified collection has fewer elements, then

the implementation iterates over the specified collection, removing from this set each element returned by the iterator, using this set's `remove` method.

Note that this implementation will throw an `UnsupportedOperationException` if the iterator returned by the `iterator` method does not implement the `remove` method.

Specified by:

`removeAll` in interface `Collection<E>`

Specified by:

`removeAll` in interface `Set<E>`

Overrides:

`removeAll` in class `AbstractCollection<E>`

Parameters:

`c` - collection containing elements to be removed from this set

Returns:

`true` if this set changed as a result of the call

Throws:

`UnsupportedOperationException` - if the removeAll operation is not supported by this set

`ClassCastException` - if the class of an element of this set is incompatible with the specified collection (`optional`)

`NullPointerException` - if this set contains a null element and the specified collection does not permit null elements (`optional`), or if the specified collection is null

See Also:

`AbstractCollection.remove(Object)`, `AbstractCollection.contains(Object)`

contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.